Entrega 2 — Greedy y Backtracking

Juan Ocampo – Cristopher Sánchez
Caso de uso: Solver de Sudoku.

1. Solución Greedy

a. Identificación de la decisión local óptima

Un algoritmo voraz (*Greedy*) construye una solución paso a paso tomando en cada iteración la decisión que parece más prometedora en ese instante (óptimo local), sin reconsiderar ni deshacer decisiones tomadas previamente. En el problema del Sudoku, la **decisión local óptima** implementada en `greedy.py` se compone de dos criterios en cada paso:

- **1. Selección de la celda más restringida (heurística MRV — *Minimum Remaining Values*):** Entre todas las celdas vacías del tablero, se calcula el conjunto de dígitos candidatos válidos (números del 1 al 9 que no se repiten en su misma fila, columna ni caja de 3x3) y se elige aquella celda vacía que tenga la **menor cantidad de candidatos disponibles**. Esta es la mejor decisión local porque una celda con un único candidato tiene certeza absoluta (0% de probabilidad de error local), y en general elegir la celda con menos opciones minimiza la incertidumbre inmediata.
- **2. Selección del valor:** Una vez elegida la celda con menos candidatos, se le asigna el primer candidato válido disponible (el menor dígito válido en orden ascendente) de manera definitiva, actualizando el tablero y repitiendo el proceso hasta llenar todas las celdas o encontrar una celda vacía sin candidatos posibles.

b. Implementación del algoritmo Greedy

La solución implementada en `src/greedy.py` se estructura en dos funciones principales:

- `candidatos(tablero, fila, columna)` — devuelve la lista de números del 1 al 9 que pueden colocarse en la posición `(fila, columna)` sin violar las reglas de fila, columna ni caja 3x3 (apoyándose en la función de validación `es_valido`).
- `resolver_greedy(tablero)` — ejecuta un ciclo iterativo (`while True`) sobre una copia del tablero. En cada iteración recorre las celdas vacías buscando la que tenga menos candidatos. Si encuentra alguna celda vacía con 0 candidatos (`not lista`), el algoritmo se atasca y retorna `(tablero, False)` inmediatamente, ya que no realiza retrocesos. Si llena todo el tablero, verifica su validez con `es_solucion_valida` y retorna `(tablero, True)`.

c. Análisis de complejidad

- **Complejidad temporal (Big O):** Sea `n` el número de celdas vacías iniciales del tablero (`n <= 81`).

1. En cada iteración del ciclo principal `while True`, el algoritmo llena exactamente una celda vacía de forma definitiva (o termina si detecta un bloqueo). Como nunca deshace una asignación, el ciclo principal se ejecuta como máximo `n` veces.
2. Dentro de cada iteración, se recorren las celdas del tablero (a lo sumo `n` celdas vacías restantes) y para cada una se invoca `candidatos(tablero, fila, columna)`. Calcular los candidatos de una celda implica probar los 9 dígitos posibles (1 al 9) y validar fila, columna y caja 3x3, lo cual tiene un costo constante `O(1)` porque las dimensiones del tablero (9x9) son fijas.
3. Por lo tanto, encontrar la celda con menos candidatos en un paso cuesta `O(n)` operaciones respecto al número de celdas vacías, y al repetirse durante a lo sumo `n` pasos, el costo total es: `T(n) = n · O(n) = O(n²)`

Dado que en un Sudoku estándar `n <= 81`, el tiempo de ejecución crece de manera polinomial cuadrática respecto a las celdas vacías (y está acotado por una constante muy pequeña en la práctica), a diferencia del costo exponencial `O(9^n)` de la fuerza bruta.

- **Complejidad espacial:** El algoritmo no utiliza recursión, por lo que no consume memoria en la pila de llamadas. Únicamente almacena una copia del tablero de trabajo de 9x9 (`O(81) = O(1)`) y listas temporales de candidatos de tamaño máximo 9 (`O(1)`). En consecuencia, la complejidad espacial adicional es **`O(1)`** (espacio constante).

d. Análisis crítico: ¿La solución Greedy es óptima para este problema?

No. El enfoque Greedy **no es óptimo ni completo** para el problema del Sudoku (no garantiza encontrar la solución aunque esta exista y sea única).

- **¿En qué casos funciona?** El algoritmo Greedy tiene éxito únicamente en tableros donde, en cada iteración, existe al menos una celda vacía con un único candidato válido (*Naked Single*), o cuando al existir empate entre varios candidatos el menor dígito resulta coincidir por azar con el valor de la solución única. En nuestras pruebas, logró resolver los casos `pequeno` (`n = 5`), `mediano` (`n = 40`) y `grande` (`n = 55`) porque tenían suficientes pistas para guiar la propagación directa.
- **¿En qué casos falla?** Falla cuando el estado del tablero llega a un punto en el que todas las celdas vacías restantes tienen 2 o más candidatos posibles (una bifurcación deductiva) y elegir localmente el menor dígito válido en una celda provoca un conflicto futuro en otra celda dependiente, dejándola sin opciones. Al no contar con un mecanismo para deshacer decisiones (*backtracking*), el algoritmo queda atrapado en un callejón sin salida.
- **Contraejemplo concreto:**

  1. Contraejemplo empírico del proyecto (`caso_extremo.txt`, `n = 64` celdas vacías, 17 pistas): Al ejecutar `resolver_greedy` sobre el caso extremo, el algoritmo toma decisiones locales con celdas que tienen múltiples candidatos, pero tras varias asignaciones deja una celda vacía con 0 candidatos válidos y termina en `0.00174 s` con estado **`sin solución`**, a pesar de que el tablero sí posee solución única (la cual sí es encontrada por Backtracking + MRV).

2. Contraejemplo ilustrativo paso a paso: Supongamos que en una misma caja 3x3 quedan tres celdas vacías `A = (0,0)`, `B = (0,1)` y `C = (1,0)` cuyos candidatos válidos actuales son:

- `Candidatos(A) = {1, 2}`
- `Candidatos(B) = {1, 3}`
- `Candidatos(C) = {2, 3}` (pero supongamos que en el resto de la fila 1 el número `3` es obligatorio en otra columna `D = (1,5)` cuyos candidatos son `{1, 3}` y donde el `1` quedará bloqueado).
- En este estado, la cantidad mínima de candidatos es 2. El algoritmo Greedy selecciona la primera celda más restringida `A = (0,0)` y elige de forma voraz su primer candidato: **`A = 1`**.
- Al fijar `A = 1`, el valor `1` deja de estar disponible en la misma caja y fila, por lo que `B = (0,1)` queda forzada a tomar **`B = 3`**, y a su vez `C = (1,0)` queda forzada a tomar **`C = 2`**.
- Sin embargo, si otra celda de la columna 0 o fila 0 necesitaba el `2` o el `3` para no quedar vacía, esa celda posterior se queda con **`0 candidatos ([])`**. Greedy retorna `False` ("sin solución") porque jamás vuelve atrás para probar la alternativa **`A = 2`**, que sí conducía a la solución global.

2. Solución con Backtracking

a. Exploración del espacio de soluciones y restricciones de poda

El algoritmo de **Backtracking** (`src/backtracking.py`) explora el espacio de soluciones como un árbol de búsqueda en profundidad (DFS). A diferencia de la fuerza bruta (que genera las `9^n` tuplas completas y valida recién al final) y del enfoque Greedy (que avanza por un solo camino sin retorno), Backtracking construye la solución celda por celda, validando las restricciones en cada paso y **retrocediendo** (*undo*) cuando una rama no puede conducir a una solución válida.

Las **restricciones que permiten podar ramas** enteras del árbol de búsqueda son:

- **1. Restricción de fila:** No se prueba ningún número que ya esté presente en la misma fila.
- **2. Restricción de columna:** No se prueba ningún número que ya esté presente en la misma columna.
- **3. Restricción de caja 3x3:** No se prueba ningún número que ya esté presente en el bloque 3x3 correspondiente.
    - *Efecto de la poda:* Si en la primera celda vacía un dígito viola cualquiera de estas tres restricciones, Backtracking descarta ese dígito inmediatamente, podando de un solo golpe las `9^(n-1)` combinaciones que comenzaban con esa asignación inválida.
- **4. Poda adicional por Heurística MRV y detección temprana de dominios vacíos:** En `src/backtracking.py` se implementaron dos variantes controladas por el parámetro `usar_mrv`:

    - **Backtracking con orden fijo (`usar_mrv = False`):** Selecciona siempre la primera celda vacía en orden de lectura (fila por fila).
    - **Backtracking + MRV (`usar_mrv = True`):** Antes de ramificar, inspecciona las celdas vacías usando máscaras de bits (`filas_usadas`, `columnas_usadas`, `cajas_usadas`). Si detecta que **cualquier** celda vacía del tablero se ha quedado sin candidatos (`mascara == 0`), **poda la rama actual de inmediato** sin seguir descendiendo. Si todas tienen candidatos, selecciona la celda con el menor número de candidatos (`mascara.bit_count()`), reduciendo drásticamente el factor de ramificación del árbol.

b. Análisis de complejidad (peor caso vs. caso promedio con poda)

- **Complejidad temporal en el peor caso (`O(9^n)`):** Desde el punto de vista teórico asintótico, para `n` celdas vacías la altura del árbol de búsqueda es `n` y en cada nivel una celda tiene a lo sumo 9 candidatos posibles. Por lo tanto, la cota superior teórica en el peor caso sigue expresándose como **`O(9^n)`** (donde cada paso realiza operaciones de máscara de bits en `O(1)` para orden fijo o `O(n)` para buscar la mejor celda en MRV).
- **Complejidad temporal en el caso promedio con poda (`O(b^n)` con `b << 9`):** En la práctica, la poda reduce el factor de ramificación efectivo `b` a un valor muchísimo menor que 9 (frecuentemente entre 1 y 2):
    - En una fila con `k` celdas vacías nunca hay 9 opciones independientes por celda, sino a lo sumo permutaciones de los dígitos faltantes, y al intersectar con las restricciones de columna y caja 3x3 la mayoría de celdas tienen solo 1, 2 o 3 candidatos válidos.
    - Con la heurística **MRV**, siempre que exista una celda con 1 solo candidato válido, el factor de ramificación en ese nivel es `b = 1` (sin ramificación ni retrocesos).
    - Esto se comprueba en nuestras mediciones empíricas: en `caso_grande` (`n = 55`), mientras la fuerza bruta tendría `9^55 ≈ 2.6 × 10^52` combinaciones y `Backtracking (orden fijo)` explora `21,622` nodos, **`Backtracking + MRV` resuelve el tablero visitando únicamente `56` nodos y realizando `0` retrocesos**.
- **Complejidad espacial (`O(n)`):** El algoritmo trabaja sobre una copia del tablero (`O(81) = O(1)`), tres arreglos de 9 enteros para las máscaras de bits de filas, columnas y cajas (`O(1)`), y utiliza recursión cuya profundidad máxima en la pila de llamadas es exactamente el número de celdas vacías `n`. Por lo tanto, la complejidad espacial es **`O(n)`**.

c. Comparación de tiempos contra las soluciones anteriores

Al ejecutar los algoritmos sobre los cuatro casos de prueba del repositorio (`datos/caso_pequeno.txt` con `n = 5`, `datos/caso_mediano.txt` con `n = 40`,

`datos/caso_grande.txt` con `n = 55` y `datos/caso_extremo.txt` con `n = 64`), se obtuvieron los siguientes tiempos y estadísticas reales:

- En **`pequeno (n = 5)`**: Fuerza Bruta tarda `0.02286 s` (probando `44,595` combinaciones), mientras que tanto Greedy (`0.00009 s`) como ambas variantes de Backtracking (`0.00008 s`, apenas `6` nodos visitados) son casi **300 veces más rápidos**.
- En **`mediano (n = 40)`**: Fuerza Bruta supera el límite de `100,000` combinaciones sin encontrar solución (`0.09973 s`). En cambio, Backtracking de orden fijo lo resuelve en `0.00014 s` (`83` nodos, `42` retrocesos), Backtracking + MRV en `0.00019 s` (`41` nodos, `0` retrocesos) y Greedy en `0.00140 s`.
- En **`grande (n = 55)`**: Aquí se aprecia la diferencia entre orden fijo y MRV: Backtracking de orden fijo necesita explorar `21,622` nodos (`21,566` retrocesos) y tarda `0.01539 s`, mientras que **Backtracking + MRV visita solo `56` nodos (`0` retrocesos) y tarda `0.00029 s`** (más de 50 veces más rápido que el orden fijo y 10 veces más rápido que Greedy con `0.00308 s`).
- En **`extremo (n = 64, 17 pistas)`**: Fuerza Bruta excede el límite, Greedy falla (`sin solución` en `0.00174 s`) y Backtracking de orden fijo también excede el límite de `100,000` nodos (`0.08425 s`). **Únicamente `Backtracking + MRV` logra resolver el caso extremo**, haciéndolo en **`0.00302 s`** tras explorar solo `678` nodos (`613` retrocesos).

3. Comparativa parcial

a. Tabla comparativa de las soluciones hasta el momento

Técnica	Complejidad Temporal	Complejidad Espacial	¿Garantiza solución óptima/válida?	Caso Pequeño (n=5)	Caso Mediano (n=40)	Caso Grande (n=55)	Caso Extremo (n=64)
Fuerza Bruta	`O(9^n)`	`O(n)`	Sí (teóricamente, inviable para `n > 7`)	`0.02286 s` (OK, 44,595 comb.)	`0.09973 s` (Excedió límite 100k)	`0.12275 s` (Excedió límite 100k)	`0.14059 s` (Excedió límite 100k)
Greedy	`O(n²)`	`O(1)`	**No** (incompleto, sin retroceso)	`0.00009 s` (OK)	`0.00140 s` (OK)	`0.00308 s` (OK)	`0.00174 s` **(Falla: sin solución)**
Backtracking (orden fijo)	`O(9^n)` peor caso (con poda)	`O(n)`	Sí (sujeto a tiempo/límite de nodos)	`0.00008 s` (OK, 6 nodos)	`0.00014 s` (OK, 83 nodos)	`0.01539 s` (OK, 21,622 nodos)	`0.08425 s` (Excedió límite 100k)
Backtracking + MRV	`O(9^n)` peor caso (poda + heurística)	`O(n)`	**Sí (resolvió 4/4 casos)**	`0.00008 s` (OK, 6 nodos)	`0.00019 s` (OK, 41 nodos)	`0.00029 s` (OK, 56 nodos)	`0.00302 s` **(OK, 678 nodos)**

b. Discusión de trade-offs (compromisos entre las técnicas)

- **1. ¿Cuál es más rápida?** En la práctica, **Backtracking + MRV** es el método más rápido en todos los tableros reales (`0.00008 s` a `0.00302 s`). Aunque teóricamente Greedy tiene menor complejidad asintótica en el peor caso (`O(n²)` vs `O(9^n)`), la representación con máscaras de bits y la reducción del árbol mediante MRV hacen que Backtracking + MRV apenas realice retrocesos y supere en velocidad real incluso a Greedy.
- **2. ¿Cuál da mejor resultado?Backtracking + MRV** ofrece el mejor resultado global: es un algoritmo **completo y exacto** que resolvió el 100% de los casos de prueba (4 de 4), incluyendo el Sudoku extremo de 17 pistas donde todos los demás métodos fallaron o agotaron el límite de cómputo. Greedy sacrifica completitud a cambio de no retroceder, por lo que falla en instancias difíciles donde es obligatorio explorar alternativas.
- **3. ¿Cuál es más fácil de implementar?Fuerza Bruta** y **Greedy** son los más sencillos de programar y comprender: Fuerza Bruta delega la generación en un producto cartesiano (`itertools.product`) y Greedy utiliza un único ciclo iterativo sin recursión ni restauración de estado. Por el contrario, **Backtracking + MRV** requiere mayor cuidado de implementación (manejo recursivo de estados, reversión exacta de cambios al retroceder y operaciones bit a bit), pero este esfuerzo adicional se traduce en una mejora dramática tanto en tiempo de ejecución como en capacidad de resolución.